User-Level Purchase Prediction from E-Commerce Behavioral Logs

Yifan Cai, Yiran Hu, Shanae Obi, Yuanqi Zhen

2026/4/27

TABLE OF CONTENTS

01 **Introduction**

goal, why important

02 **Our Dataset**

REES46 Multi-Category Store e-commerce behavioral dataset (October 2019)

03 **EDA**

Dataset analysis

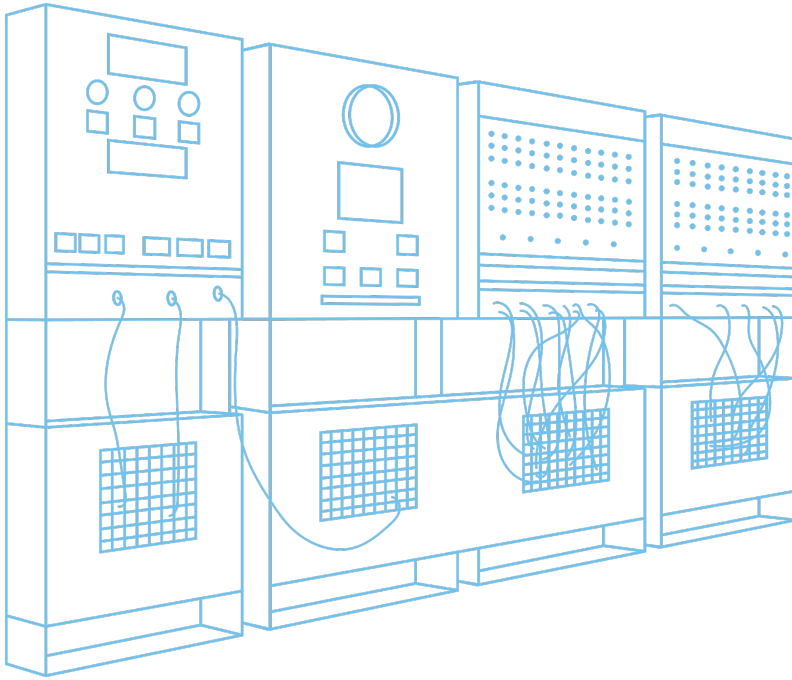
04 **3 Models**

LR; XGBoosting, MLP

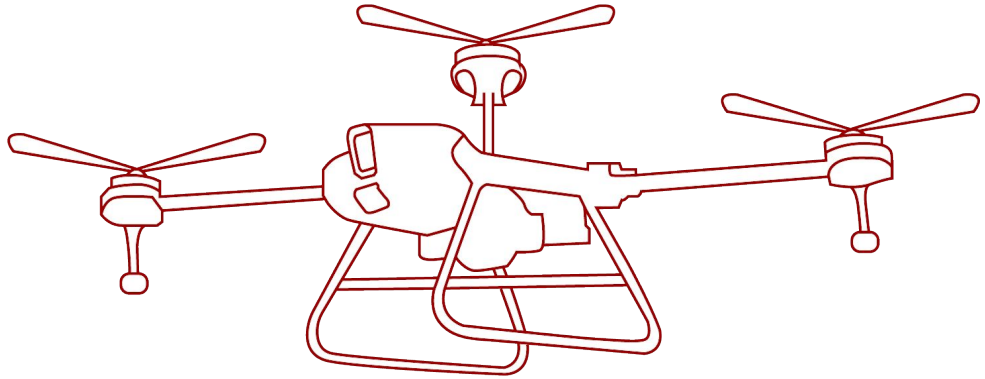
05 **Conclusion**

insights

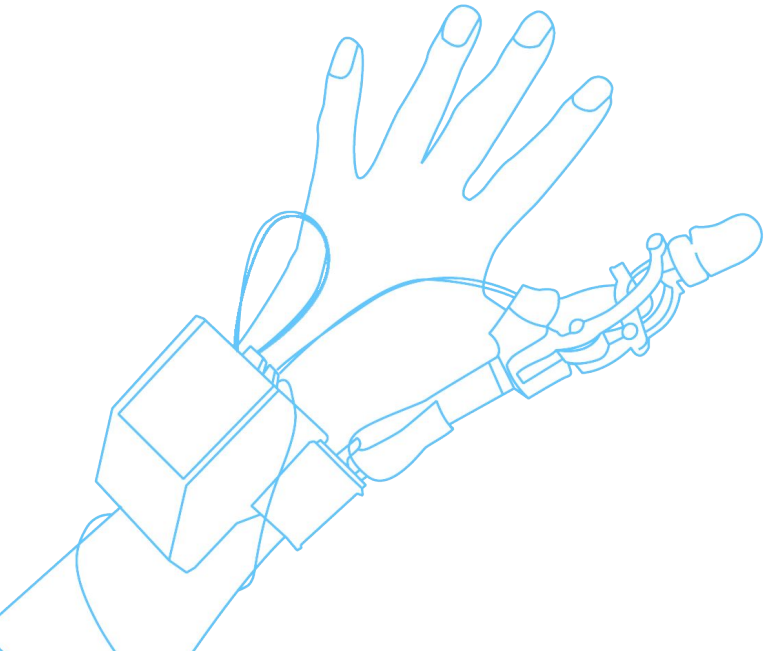
Penn Engineering
University of Pennsylvania



Penn Engineering
University of Pennsylvania



Introduction

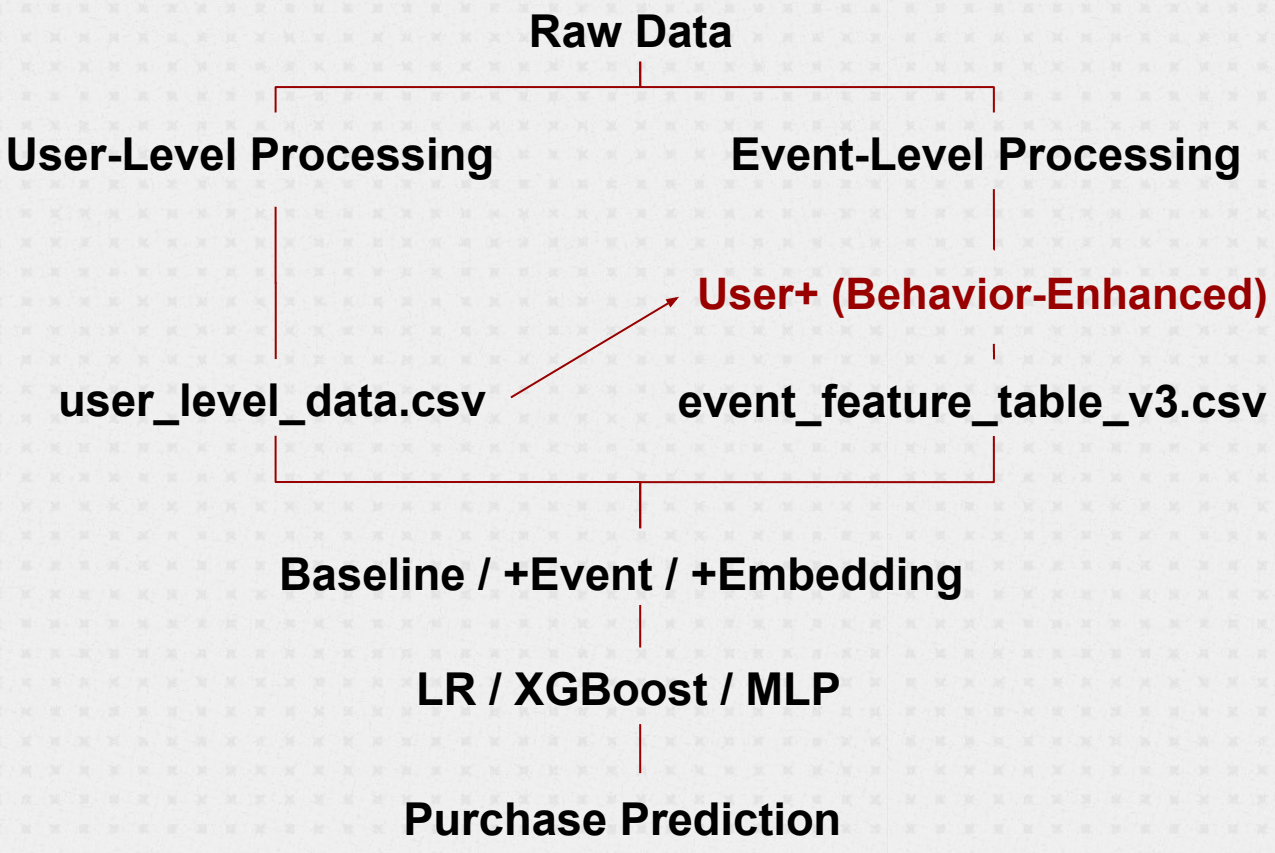


Our target

- Predict whether a user will make a purchase
- Binary classification at the user level
- Based on historical interaction behavior
  (view, cart, purchase)

Why it matters

- Core task in recommendation & advertising systems
- Helps improve conversion rate & user targeting
- Widely used in real-world e-commerce platforms



Step 1 : Baseline

User-Level Features

Step 2 : Baseline + Event Features

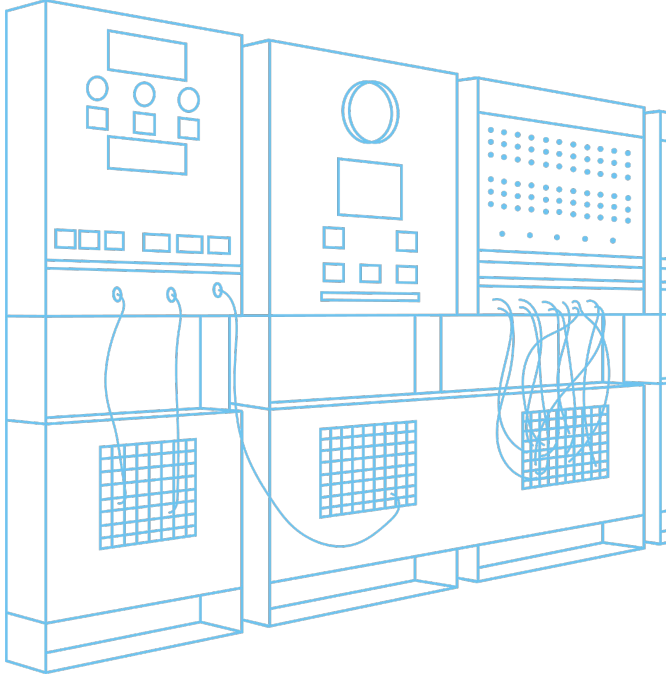
User-Level Features + Event-Derived Features

Step 3: Baseline + Event + Embedding

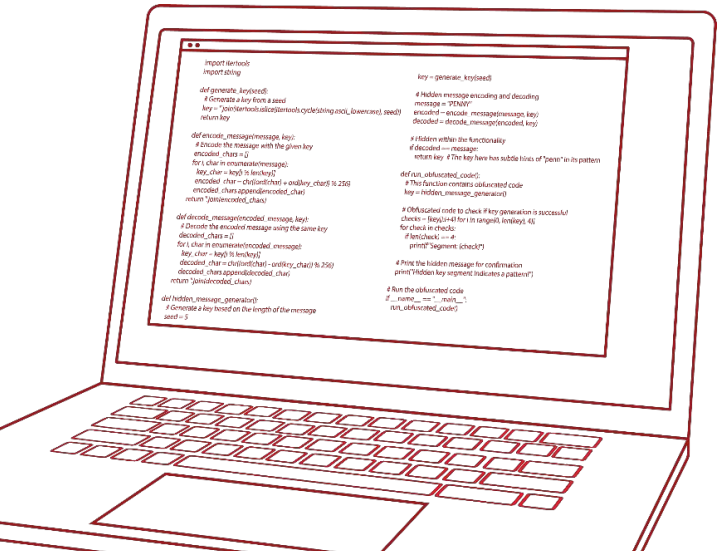
User-Level Features + Event-Derived Features + Embedding Features

For example, if users interacting with “electronics” tend to have a higher purchase rate than those in “clothes,” Because the former appears more likely to be the subject of purposeful or urgent purchases.

Penn Engineering
UNIVERSITY of PENNSYLVANIA



Our Dataset



```
import itertools
import string

def generate_key(seed):
    # Generate a key from a seed
    key = ''.join(itertools.islice(itertools.cycle(string.ascii_lowercase), seed))
    return key

def encode_message(message, key):
    # Encode the message with the given key
    encoded_chars = []
    for i, char in enumerate(message):
        key_char = key[i % len(key)]
        encoded_char = chr((ord(char) + ord(key_char)) % 256)
        encoded_chars.append(encoded_char)
    return ''.join(encoded_chars)

def decode_message(encoded_message, key):
    # Decode the encoded message using the same key
    decoded_chars = []
    for i, char in enumerate(encoded_message):
        key_char = key[i % len(key)]
        decoded_char = chr((ord(char) - ord(key_char)) % 256)
        decoded_chars.append(decoded_char)
    return ''.join(decoded_chars)

def hidden_message_generator():
    # Generate a key based on the length of the message
    seed = 5
    key = generate_key(seed)

    # Hidden message encoding and decoding
    message = "PENNY"
    encoded = encode_message(message, key)
    decoded = decode_message(encoded, key)

    # Hidden within the functionality
    if decoded == message:
        return key # The key have has subtle hints of "penn" in its pattern

def run_obfuscated_code():
    # This function contains obfuscated code
    key = hidden_message_generator()

    # Obfuscated code to check if key generation is successful
    checks = [key[i]==41 for i in range(0, len(key), 4)]
    for check in checks:
        if len(check) == 4:
            print(f"Segment: {check}")

    # Print the hidden message for confirmation
    print("Hidden key segment indicates a pattern!")

# Run the obfuscated code
if __name__ == '__main__':
    run_obfuscated_code()
```

Raw Data

REES46 Multi-Category Store · Kaggle · October 2019

42M+	3M+	4	9
interaction records	unique users	event types	raw features

Key Fields per Event Row:

event_time	Timestamp of interaction	**brand**	Product brand (may be missing)
event_type	view / cart / remove / purchase	**price**	Price at interaction time
product_id / category_id	Product & category identifiers	**user_id / user_session**	User & session identifiers

Three-Layer Data Architecture

From raw logs → structured features → enriched behavioral representation

① Event-Level

~42M rows · one row per user action

- Parse timestamps to UTC; sort by user_id + event_time
- Fill missing brand & category_code with 'unknown'
- Split category_code into 3 hierarchical levels
- Consolidate rare categories (<0.01% frequency) → 'other'
- No aggregation — preserves full behavioral sequence

② User-Level (Baseline)

~3M rows · one row per user

- Aggregate views, carts, purchases, avg/max price
- Binary label: purchased = 1 if any purchase event
- Smoothed conversion rates (Laplace +1 denominator)
- Temporal features: avg hour, hour_std, active_days
- Diversity: num_categories, num_products; binary flags

③ User+ (Behavior-Enhanced)

~3M rows · richer sequential signals

- Session structure: num_sessions, session length stats
- Temporal dynamics: active_duration, inter-event gaps
- Funnel transitions: view→cart→purchase conversion rates
- Repeated-interest: re-views on same product
- Category preference entropy + price distribution features

Design Rationale

Why build two complementary feature tables instead of one?

Evaluate Incrementally

Keeping User-Level and User+ separate lets us measure exactly how much behavioral richness improves predictions over the baseline.

No Aggregation at Event Level

Event-level data is never directly fed to models — it avoids data leakage and preserves full interaction sequences for clean feature engineering.

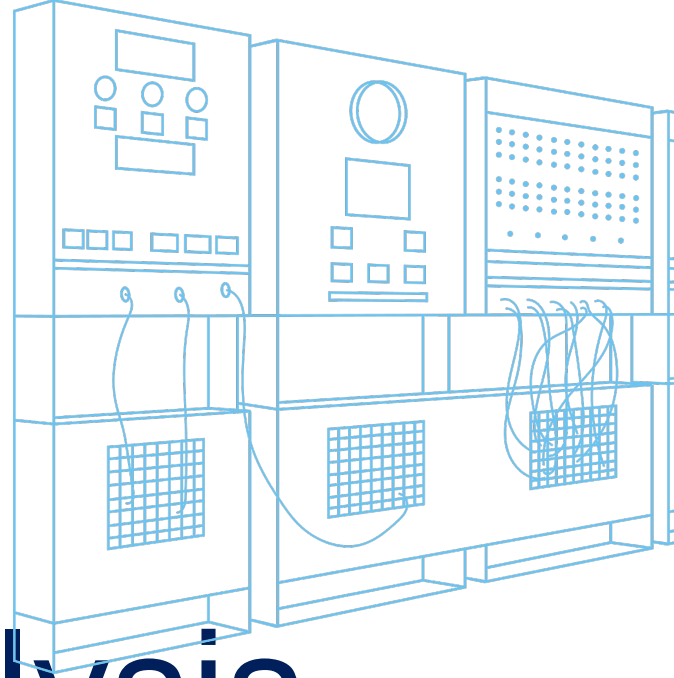
Class Imbalance is a Modeling Problem

Only ~11.5% of users purchased. We don't fix this in preprocessing — instead we use scale_pos_weight (XGBoost) and weighted BCE loss (MLP).

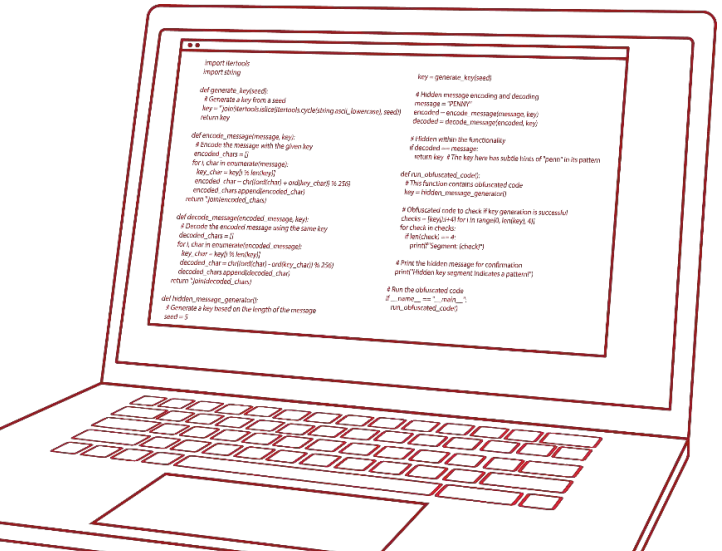
Modular & Interpretable

Each layer has a clear role. This makes the pipeline easier to debug, compare across models, and extend with new features.

Penn Engineering
UNIVERSITY of PENNSYLVANIA

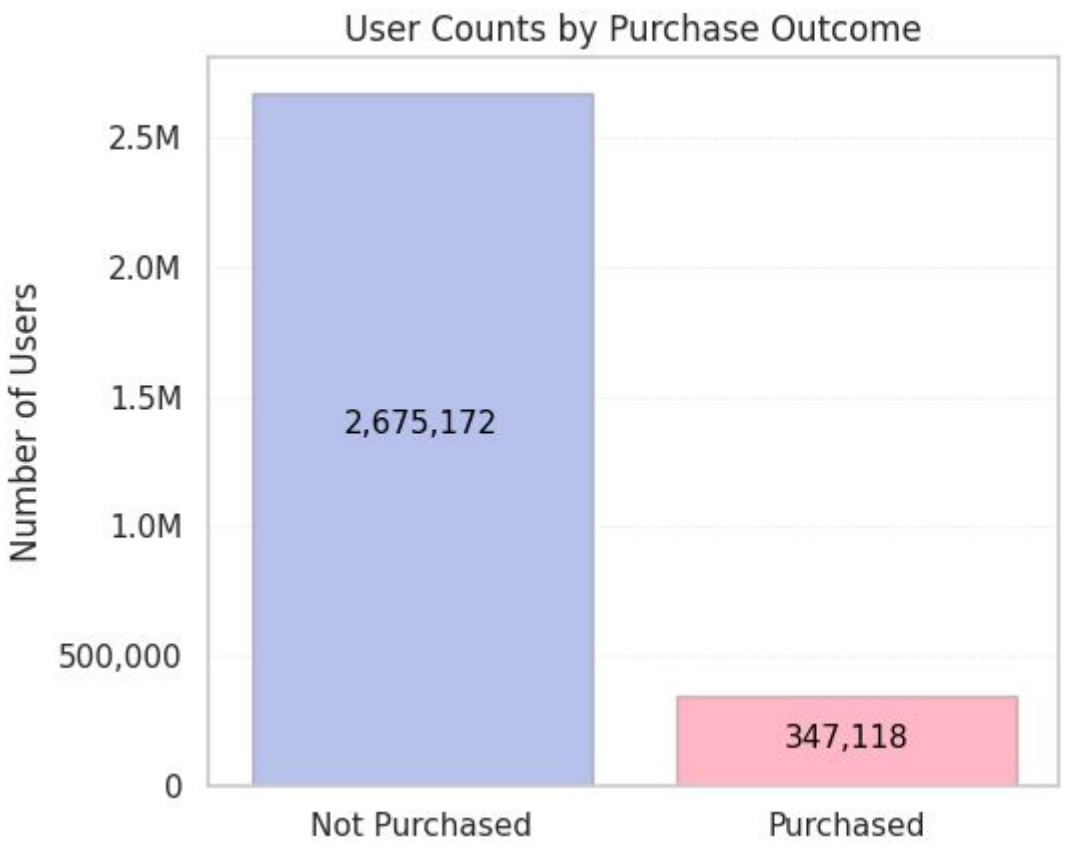


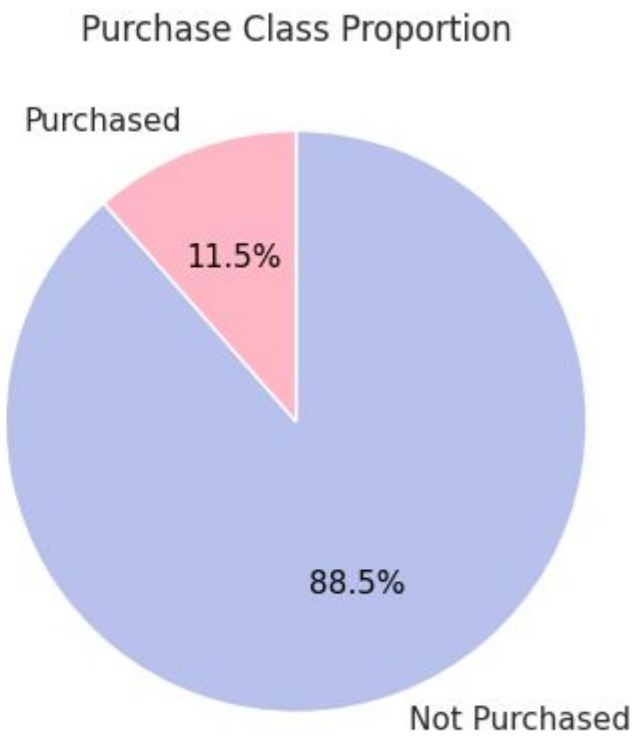
Exploratory Data Analysis



Target Distribution

Class Imbalance



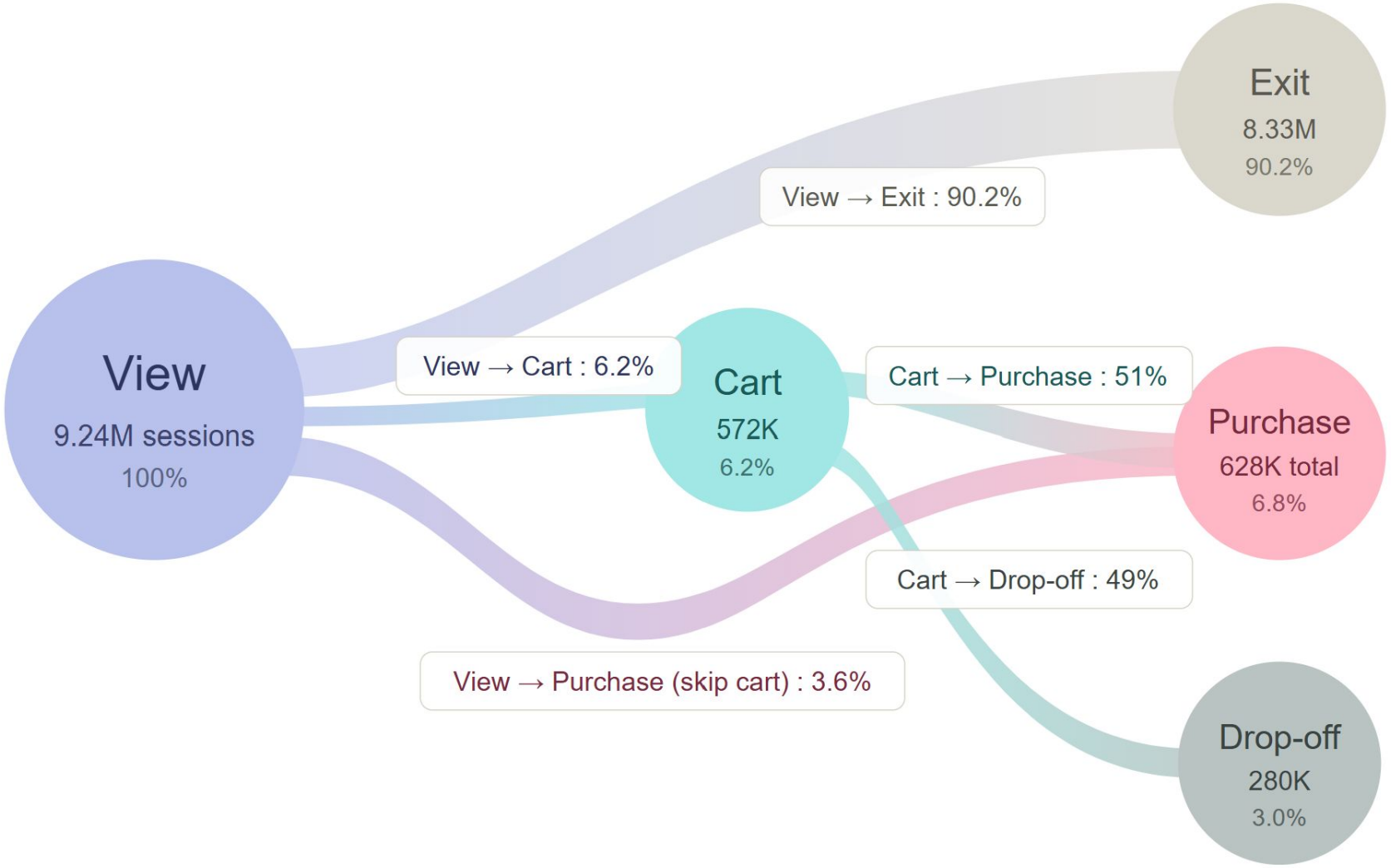


From Logs to Purchase Signals

Activity Level	How much each user interacts?
Price Exposure	How prices appear across user actions?
Category Scope	How broad or focused their browsing is?
Timing patterns	When, and how consistently users interact?
Behavior Pathways	How users move to a purchase?

Behavioral Pathways

Key Behaviors: View - Cart - Purchase



Event behavior features trace these paths:

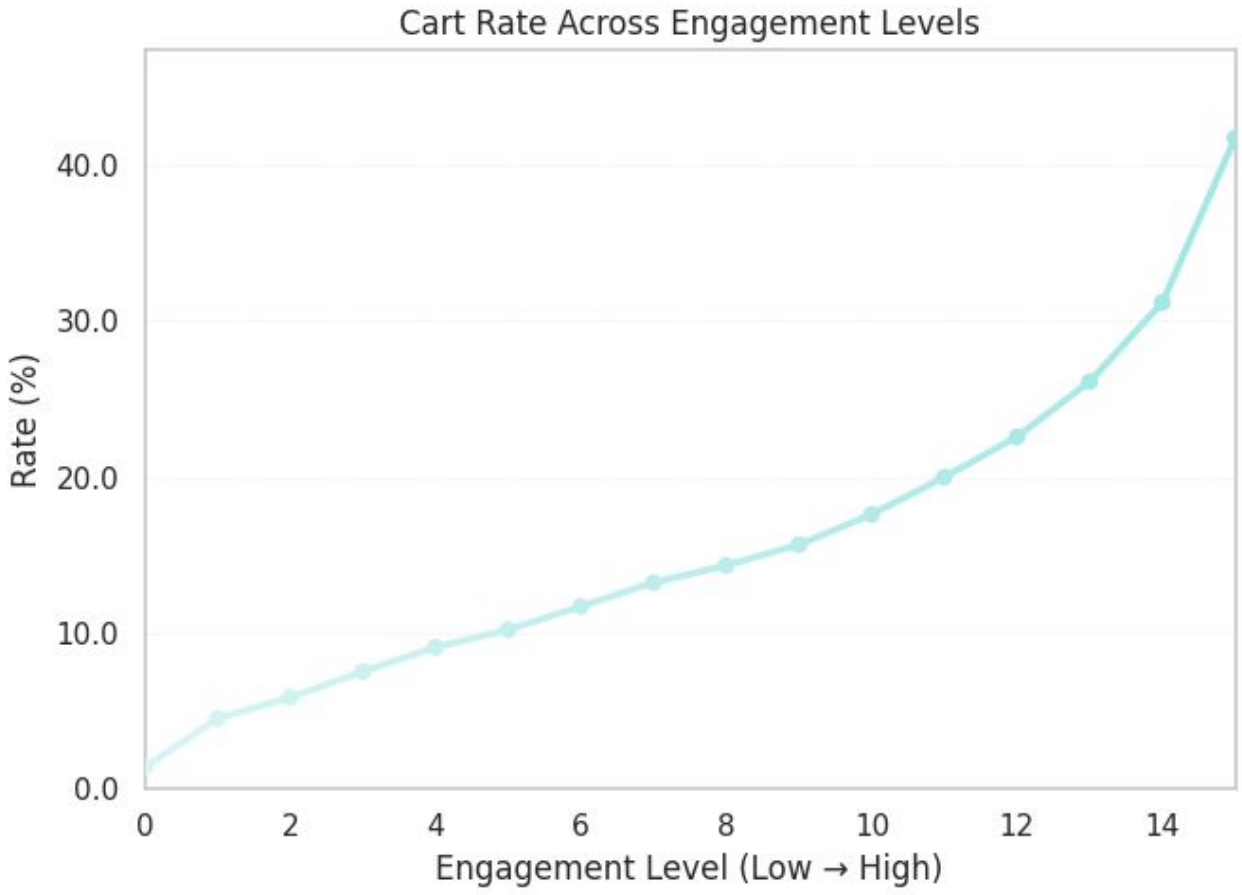
Most sessions stop at **viewing**

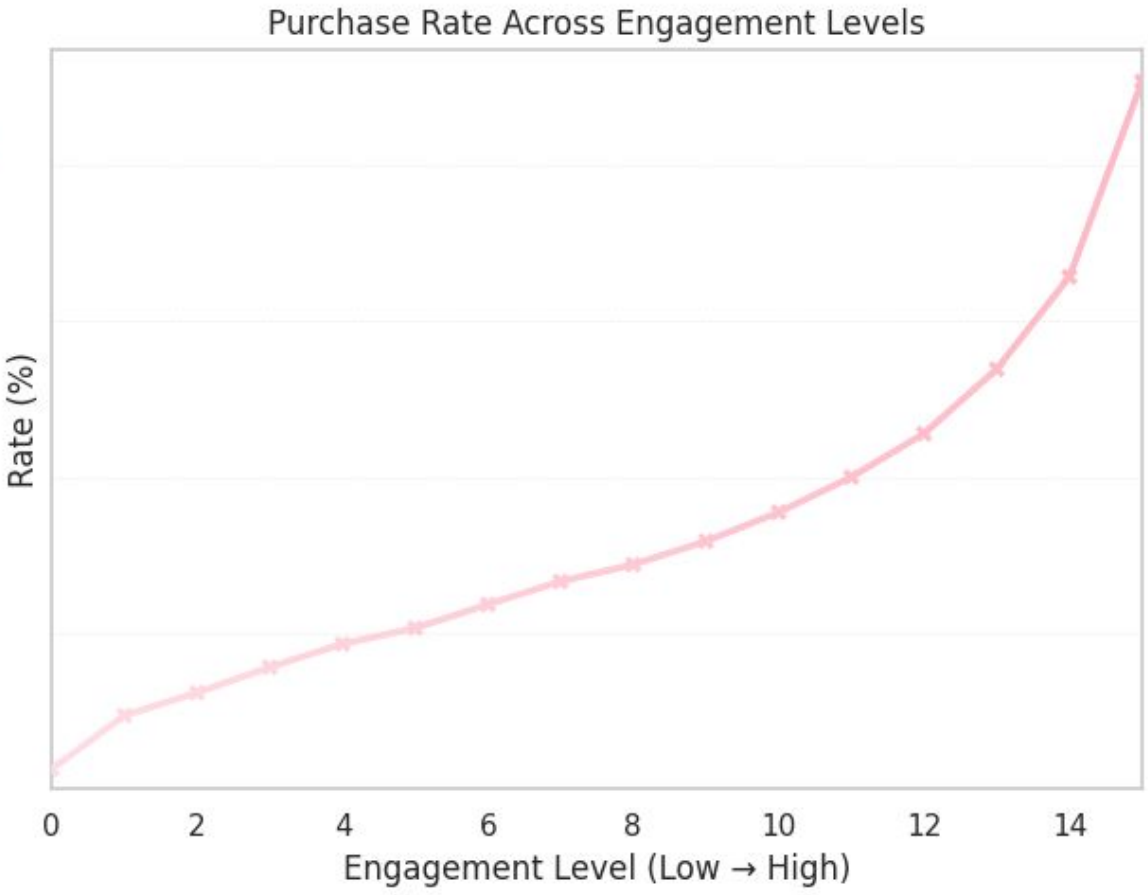
About **half** of **carts** become purchases

Some users purchase **without carting**

Activity Matters

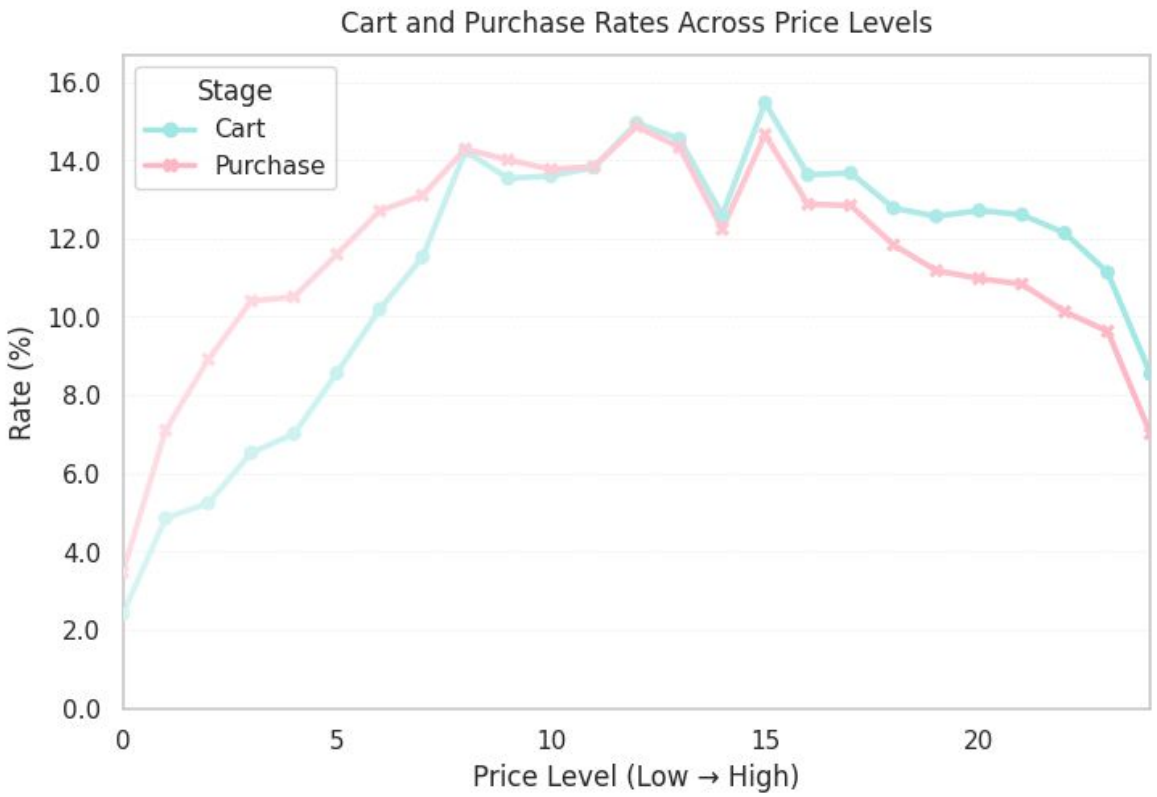
Cart and Purchase Outcome by Activity

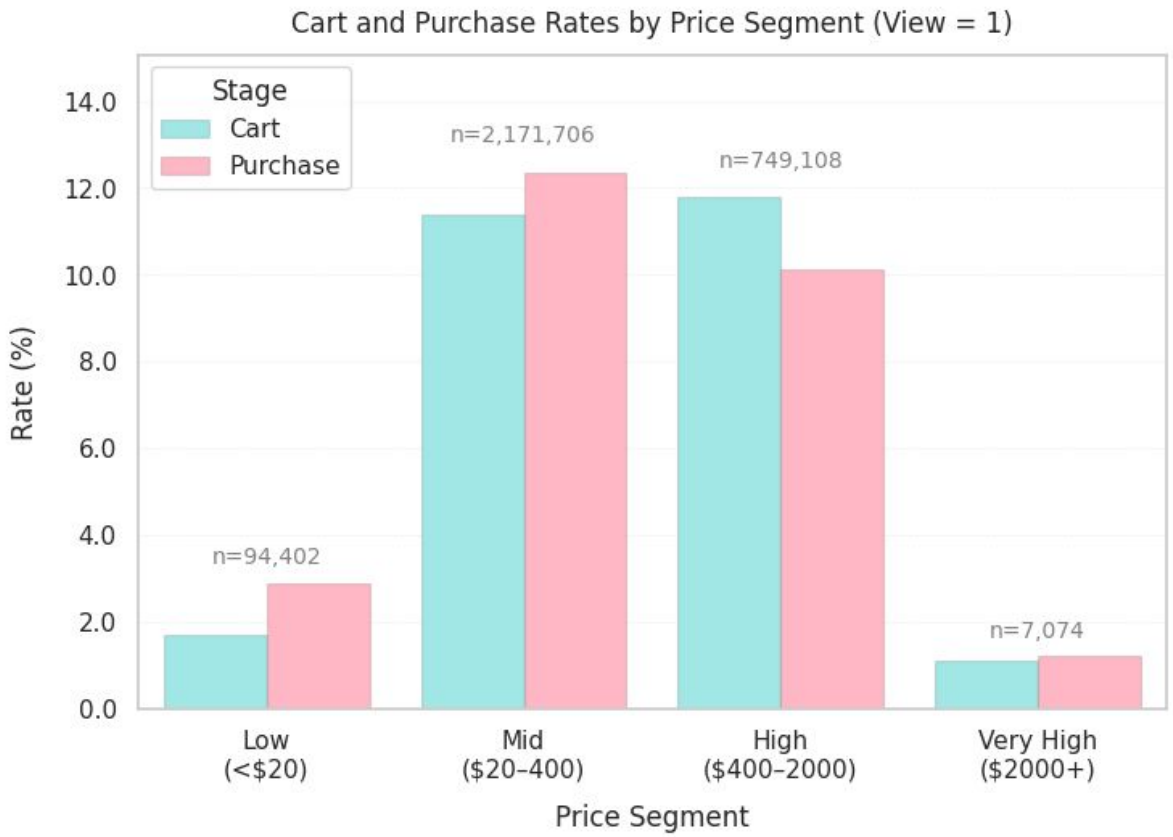




Price Matters

Cart and Purchase Outcome by Price

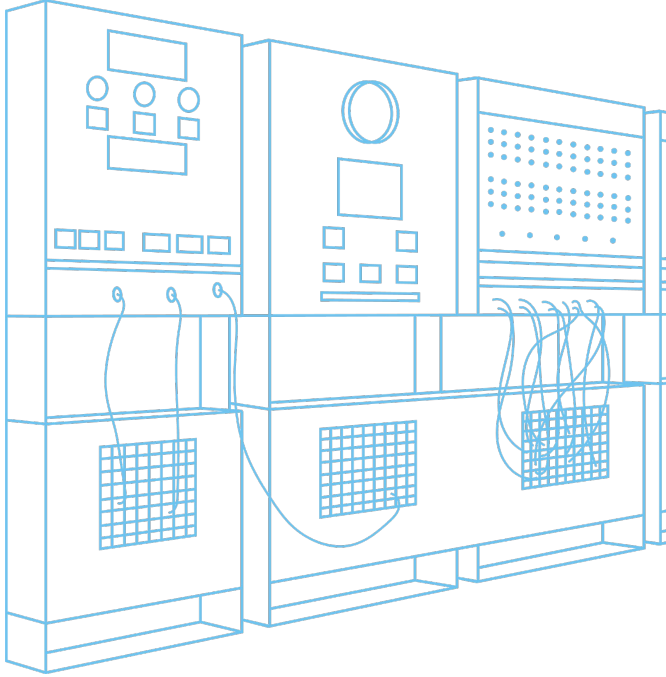




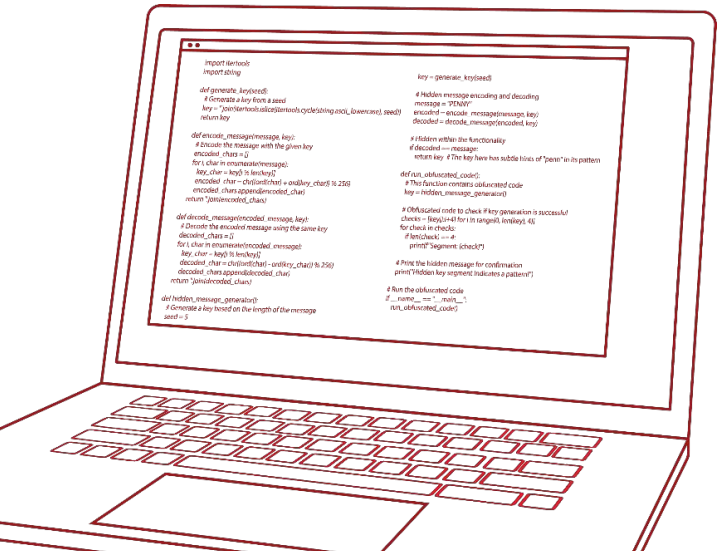
From Logs to Purchase Signals

Activity Level	How much each user interacts?
Price Exposure	How prices appear across user actions?
Category Scope	How broad or focused their browsing is?
Timing patterns	When, and how consistently users interact?
Behavior Pathways	How users move to a purchase?

Penn Engineering
University of Pennsylvania



Models



Three Steps of our models

Step 1 : Baseline

User-Level Features

Step 2 : Baseline + Event Features

event-based behavioral features, which capture user intent.

Step 3: Baseline + Event + Embedding

introduces category information, to model category-level effects.

Model-specific encoding strategy:
LR / XGBoost → One-hot
MLP → Embedding

LR

V.S.

XGBoost

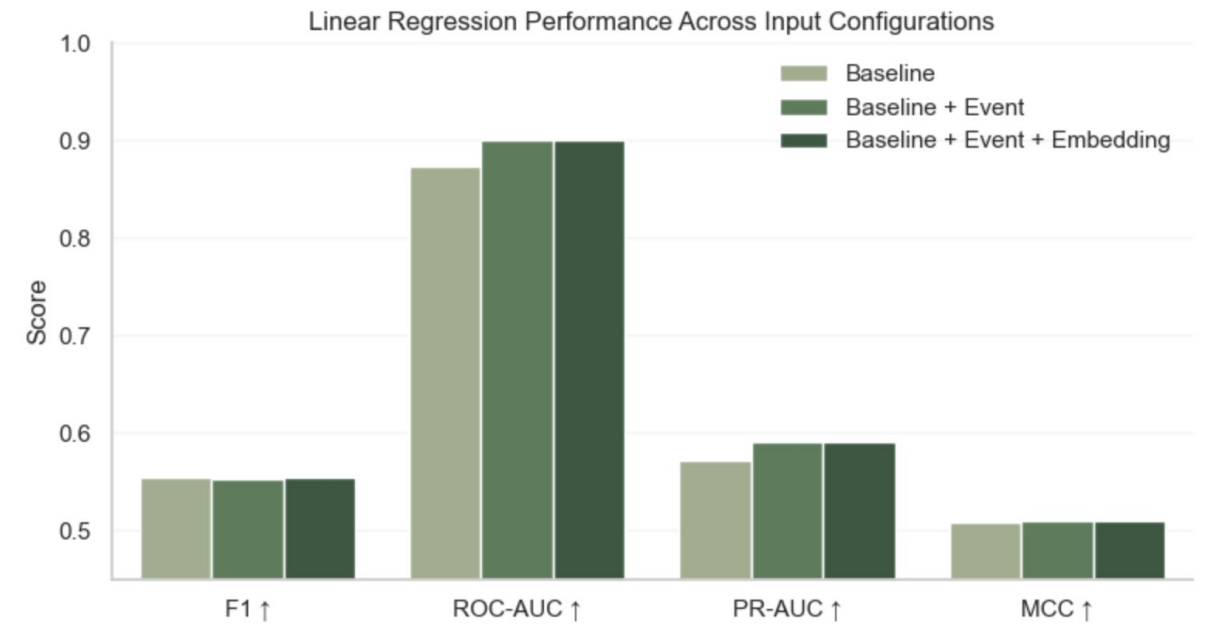
V.S.

MLP

Linear Regression

- **Linear model**
- **Interpretable baseline**

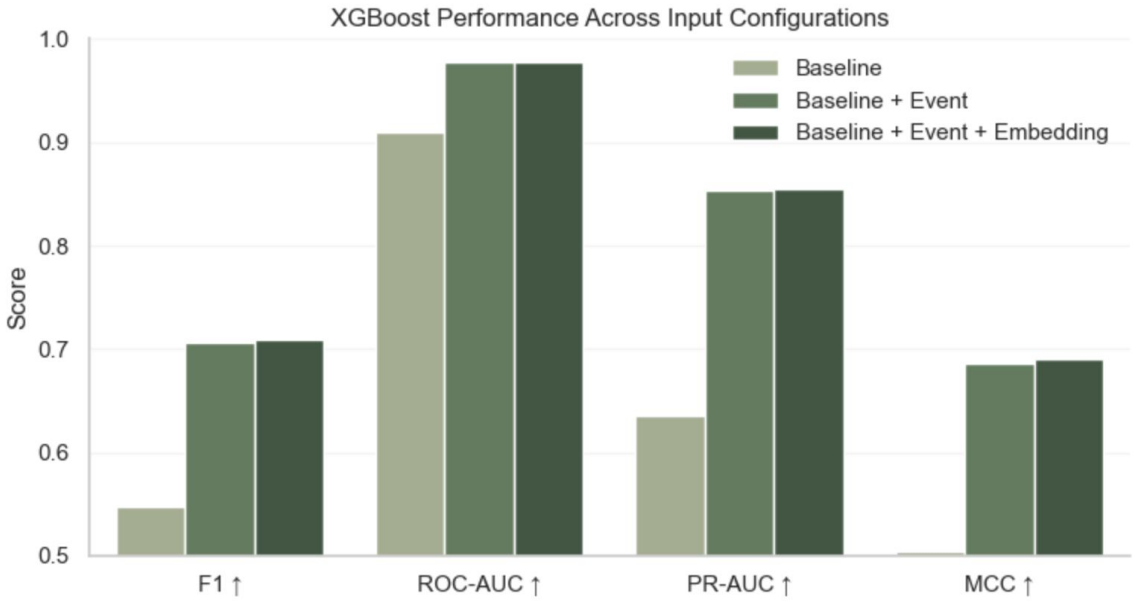
- Baseline provides reasonable performance
- Adding event features improves performance
- Limited ability to capture complex patterns



XGBoost

- **Tree-based model**
- **Captures non-linear interactions**

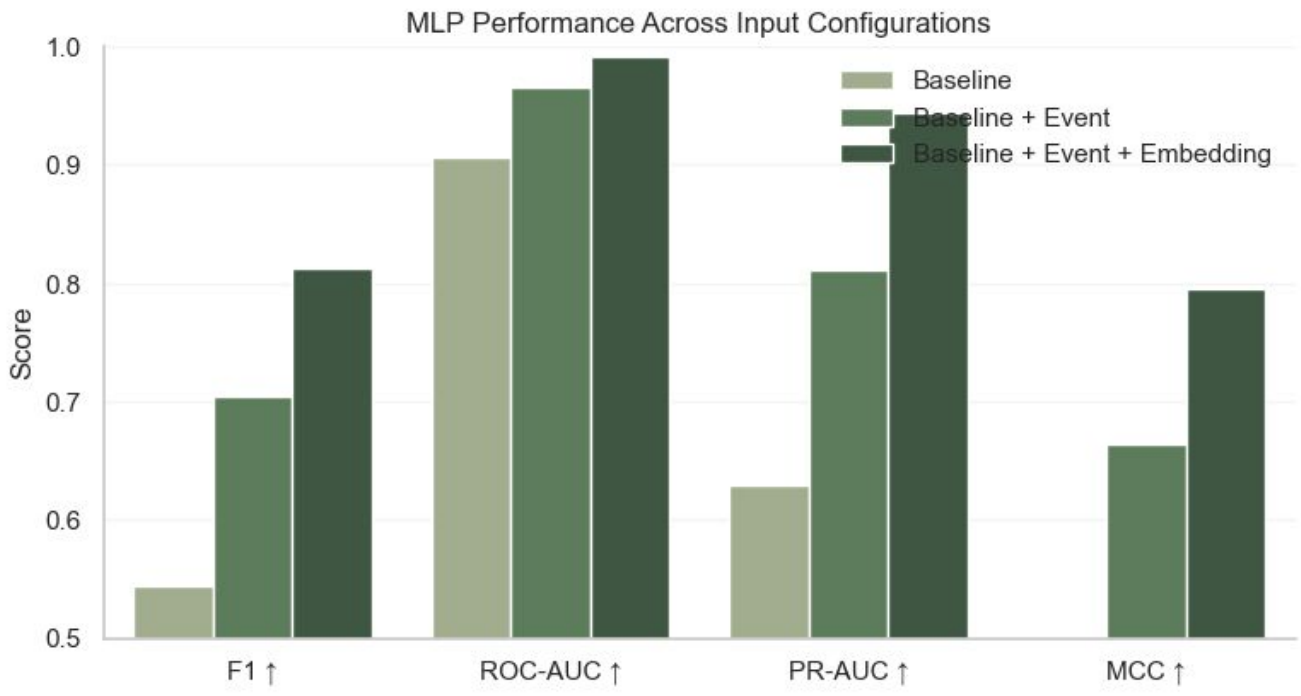
- Significant improvement over linear models
- Strong gains from event-based features
- Limited additional value from category features
- Effective for tabular behavioral data



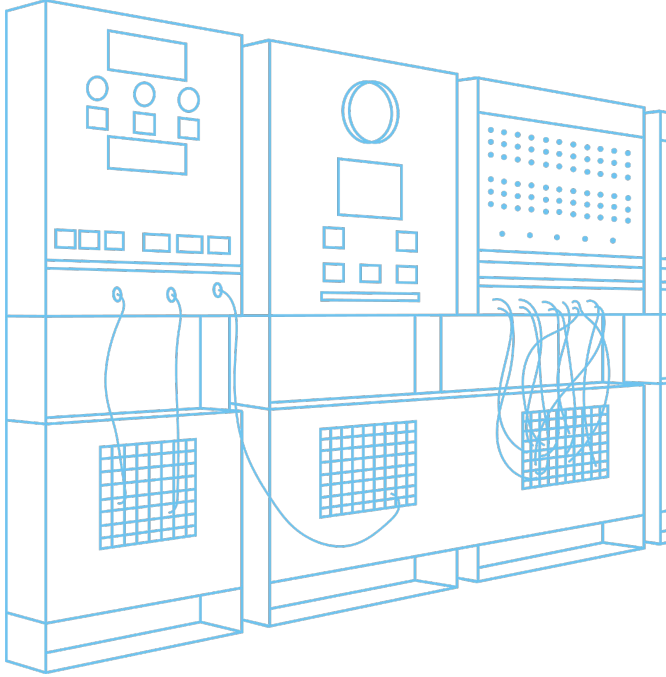
MLP (Neural Network)

- **Neural network**
- **Supports learnable embeddings**

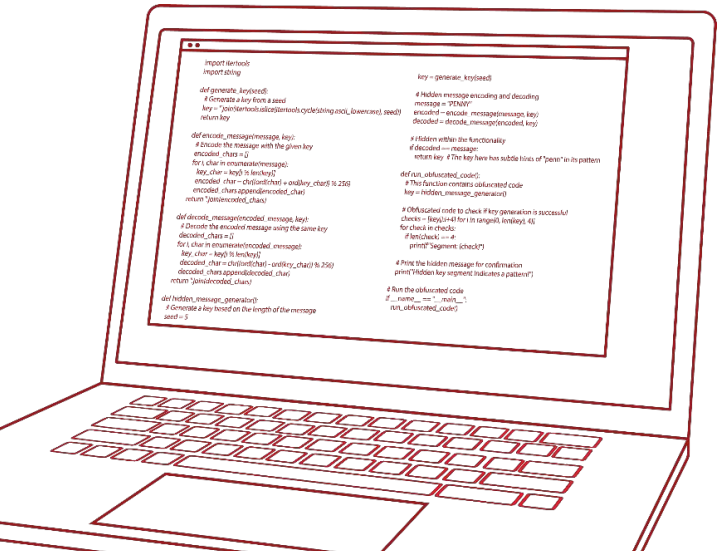
  - Best overall performance among models
  - Strong gains from event-based features
  - Additional improvement from category embeddings



Penn Engineering
UNIVERSITY of PENNSYLVANIA

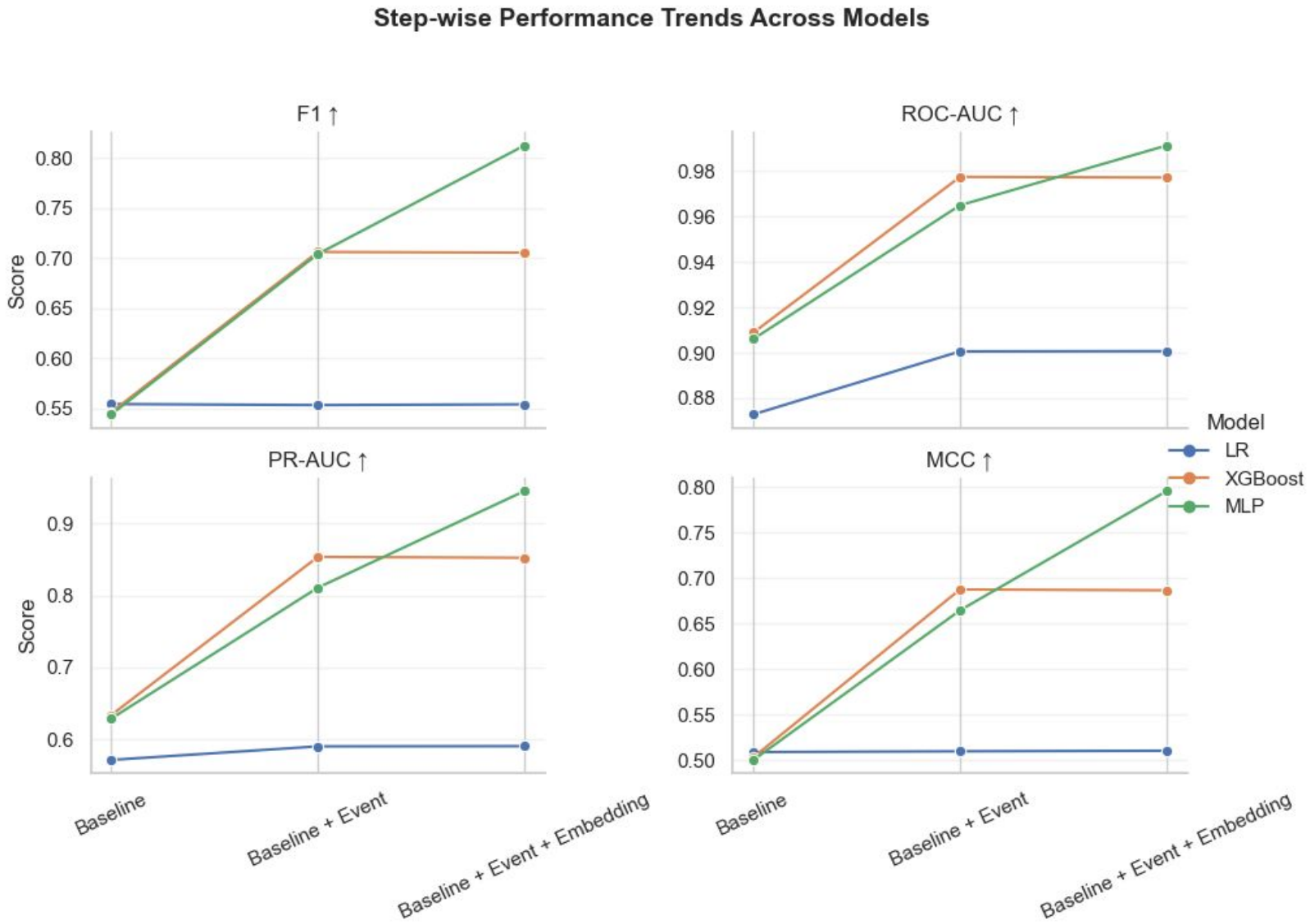


Insights



```
import itertools
import string

def generate_key(seed):
    # Generate a key from a seed
    key = ''.join(itertools.islice(itertools.cycle(string.ascii_lowercase), seed))
    return key

def encode_message(message, key):
    # Encode the message with the given key
    encoded_chars = []
    for i, char in enumerate(message):
        key_char = key[i % len(key)]
        encoded_char = chr(ord(char) + ord(key_char)) % 256)
        encoded_chars.append(encoded_char)
    return ''.join(encoded_chars)

def decode_message(encoded_message, key):
    # Decode the encoded message using the same key
    decoded_chars = []
    for i, char in enumerate(encoded_message):
        key_char = key[i % len(key)]
        decoded_char = chr(ord(char) - ord(key_char)) % 256)
        decoded_chars.append(decoded_char)
    return ''.join(decoded_chars)

def hidden_message_generator():
    # Generate a key based on the length of the message
    seed = 5
    key = generate_key(seed)

    # Hidden message encoding and decoding
    message = 'PENNY'
    encoded = encode_message(message, key)
    decoded = decode_message(encoded, key)

    # Hidden within the functionality
    if decoded == message:
        return key # The key have has subtle hints of "penn" in its pattern

def run_obfuscated_code():
    # This function contains obfuscated code
    key = hidden_message_generator()

    # Obfuscated code to check if key generation is successful
    checks = [key[i]==41 for i in range(0, len(key), 4)]
    for check in checks:
        if len(check) == 4:
            print("Segment: {check}")

    # Print the hidden message for confirmation
    print("Hidden key segment indicates a pattern!")

# Run the obfuscated code
if __name__ == '__main__':
    run_obfuscated_code()
```

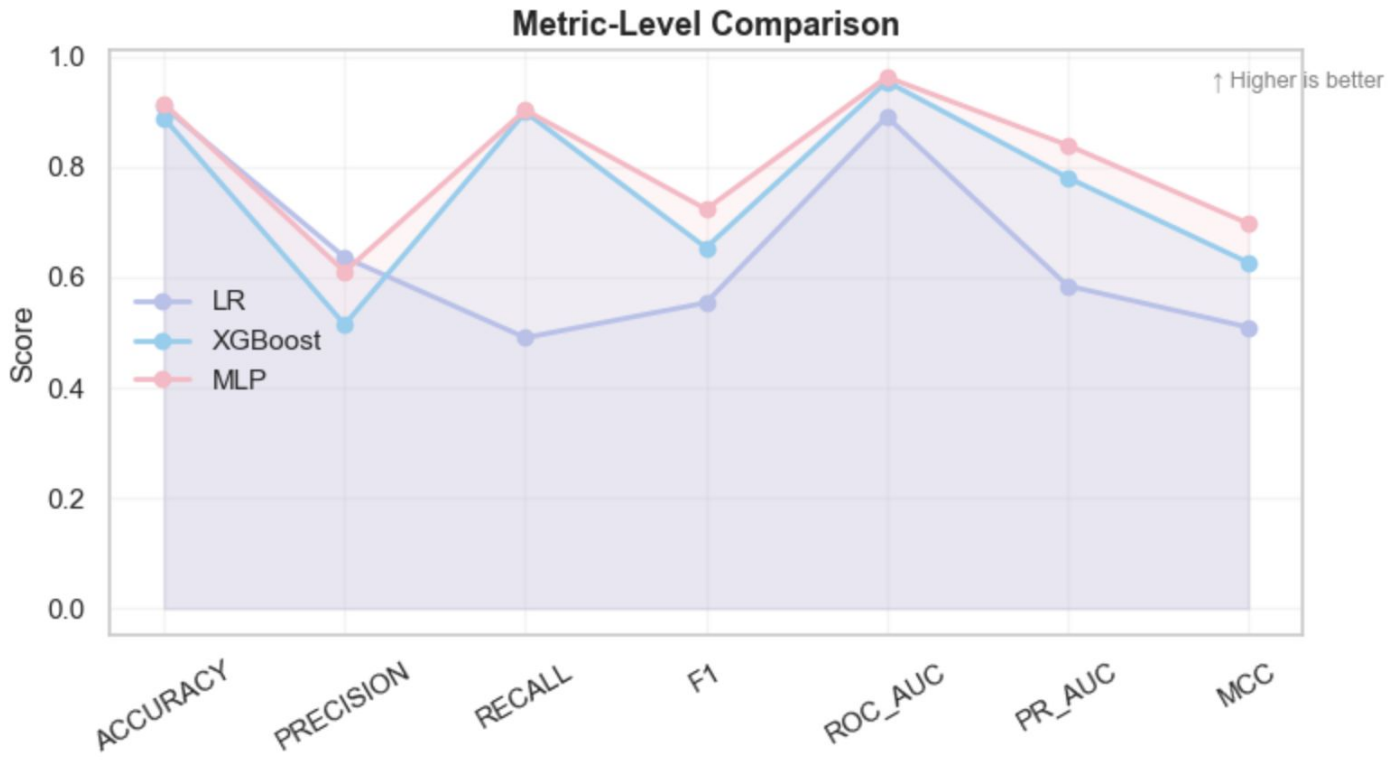
Feature Contribution Analysis



- Step 2 provides the largest performance gains
- Step 3 further improves MLP through embeddings
- Behavioral features achieve strong performance
- MLP effectively leverages embedding features

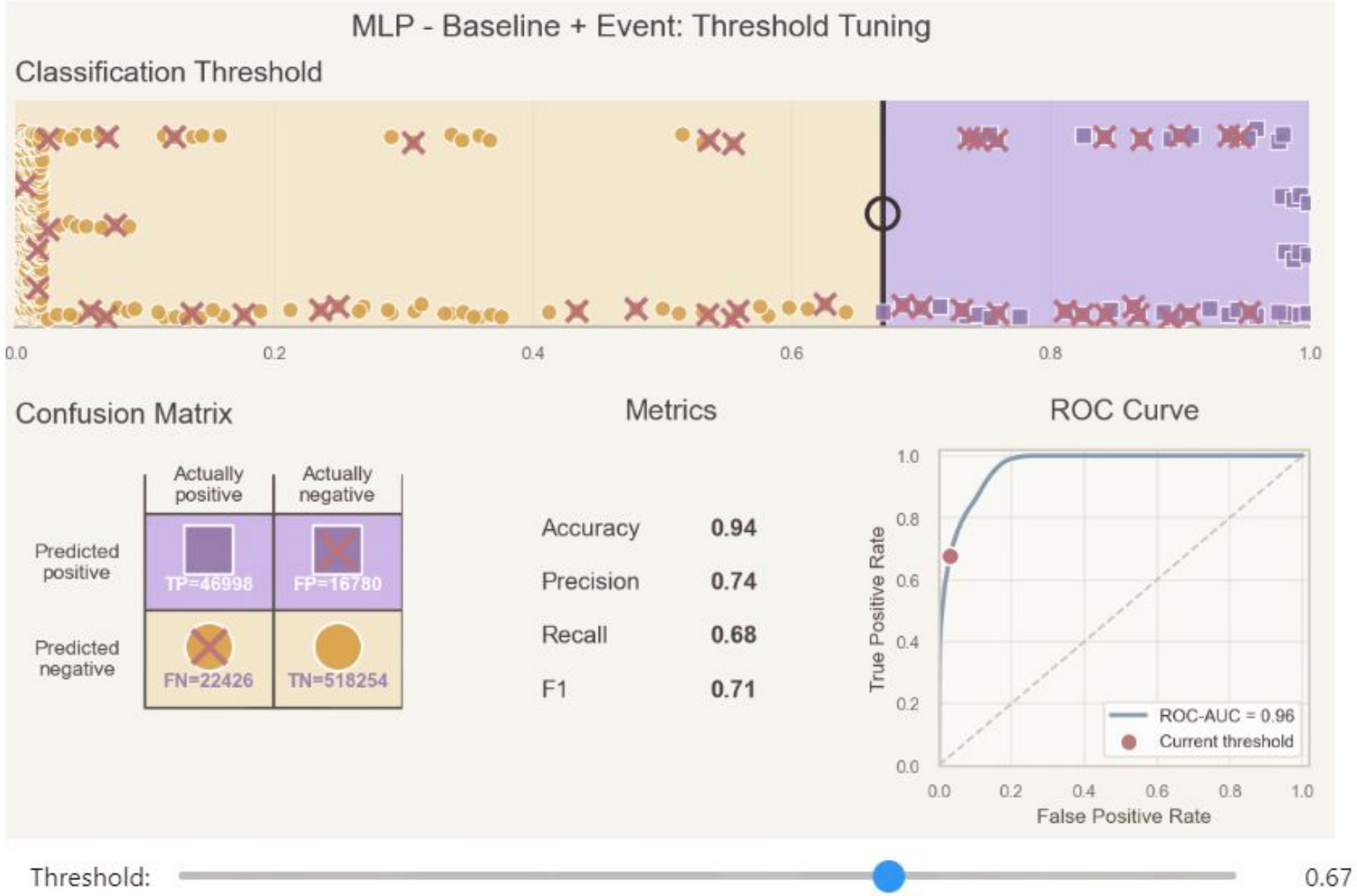
Model Performance Comparison Across Metrics

- MLP performs best across most metrics
- XGBoost is strong but slightly below
- Linear Regression performs worst
- High ROC-AUC but lower F1 shows classification difficulty



Threshold Tuning for Imbalanced Data

- Optimal threshold is around 0.7
- Higher threshold reduces false positives
- Model makes more conservative positive predictions
- Suitable for imbalanced purchase data



Challenges, Limitations & Future Work

Penn Engineering
University of Pennsylvania

Challenges

- Highly imbalanced dataset makes prediction and evaluation more difficult
- Behavioral signals dominate for simpler models, making additional gains harder

Limitations

- Coarse category representation limits full utilization of category information
- Lack of fine-grained user–category interaction features

Future Work

- Incorporate fine-grained category-level behavioral features
- Model multi-category preferences and user interaction sequences
- Further improve representation learning for user–category interactions

Penn Engineering
University of Pennsylvania

Thank You

Yifan Cai, Yiran Hu, Shanae Obi, Yuanqi Zhen

2026/4/27